

GATE CS Revision Notes

Database Management Systems

WHAT IS A DBMS ?

A DBMS is software that stores, manages and queries large amounts of related data in a safe, shared way. It hides the messy file details and gives us a clean, high-level view of the data.

File system vs DBMS :-

- Files : data scattered, much redundancy
- Files : no easy query, weak security
- DBMS : one controlled store, less redundancy
- DBMS : queries, ACID, backup, many users

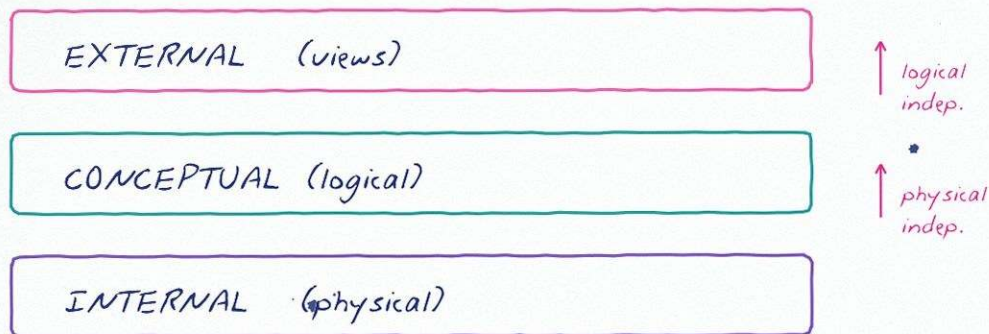
Why it matters :-

The DBMS removes data redundancy and inconsistency. It enforces rules, controls concurrent access, and recovers after a crash. These four jobs are the heart of the whole subject.

THREE-SCHEMA ARCHITECTURE

Idea :-

ANSI-SPARC splits the database into three levels so that a change at one level does not disturb the others. This separation is what gives us data independence.



The three levels :

- External : user views, what each user sees
- Conceptual : whole logical design (tables)
- Internal : physical storage, files, indexes

Data independence :-

Logical independence means we can change the conceptual schema, like adding a column, without rewriting the user views. Physical independence means we can change storage or indexes without touching the logical schema. Logical independence is harder to achieve.

KEYS

Concept :-

A key is a set of attributes that identifies rows of a relation. Keys stop duplicate rows and link tables together, so almost every design and query idea depends on them.

Types of keys :

- Super key : any attr set that is unique
- Candidate key : minimal super key
- Primary key : one chosen candidate key
- Composite key : two or more attributes

Foreign key :-

A foreign key is an attribute in one table that refers to the primary key of another table. It enforces referential integrity, so you cannot point to a row that does not exist.

candidate $\equiv$ minimal super key

no extra attr

- prime attribute : part of ANY candidate key

ER MODEL : ENTITIES

Definition :-

The ER model draws the mini-world as entities and the relationships between them. It is the first step of design, done before we make any tables.

Entity & attributes :

- Entity : a real thing (Student, Course)
- Attribute : a property (name, age)

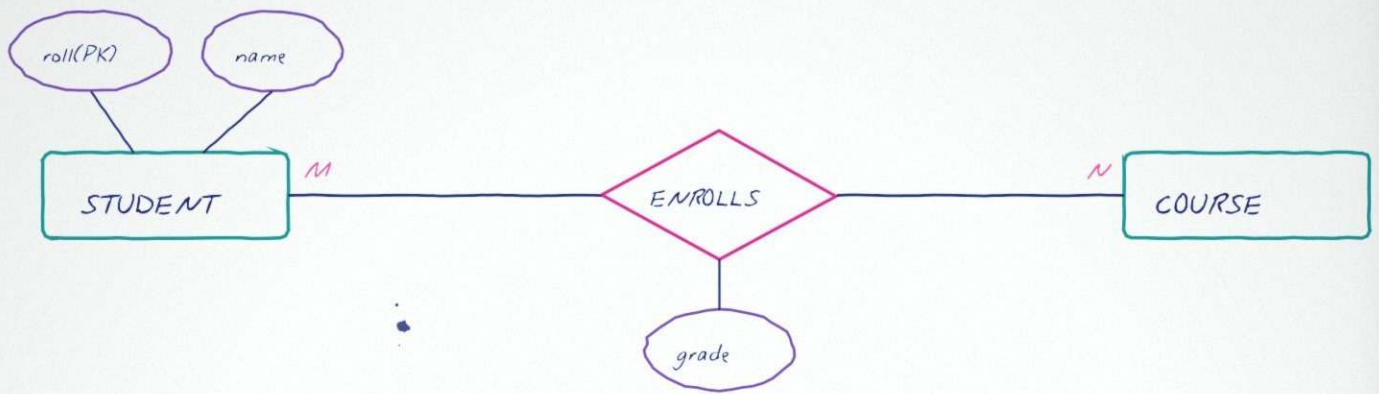
Attribute types :-

- Simple vs Composite (name → first, last)
- Single vs Multi-valued (phone numbers)
- Stored vs Derived (age from DOB)
- Key attribute : underlined, unique

Shapes :-

Rectangle is an entity, oval is an attribute, diamond is a relationship, and a double shape means weak entity or multivalued. Learn the shapes; GATE loves ER-to-table questions.

ER : RELATIONSHIPS



Cardinality :-

→ 1:1 , 1:N , M:N (max mapping)

Cardinality says how many entities on one side link to the other side. It decides where the foreign key goes when we map to tables.

Participation :-

→ Total : every entity must take part (double line)

→ Partial : taking part is optional

Weak entity :-

A weak entity has no key of its own. It depends on a strong (owner) entity and is identified by the owner's key plus its own partial key. It always has total participation.

ER $\rightarrow$ RELATIONAL MAPPING

Rules :-

- $\rightarrow$ Strong entity $\rightarrow$ table, its key = PK
- $\rightarrow$ Multivalued attr $\rightarrow$ its own table
- $\rightarrow$ Weak entity $\rightarrow$ table + owner PK in key

Relationships :-

- $\rightarrow$ 1:1 $\rightarrow$ add FK on either side
- $\rightarrow$ 1:N $\rightarrow$ FK on the N (many) side
- $\rightarrow$ M:N $\rightarrow$ new table with both PKs

Why the M:N table :-

A many-to-many link cannot fit inside either entity table, so we make a separate relationship table. Its primary key is the pair of the two entity keys, plus any relationship attributes like grade.

M:N $\rightarrow$ 3 tables total

never merge M:N

RELATIONAL MODEL

Definition :-

Data is stored as relations, that is, simple two-dimensional tables of rows* and columns. Its clean maths lets us reason about queries and correctness.

Terms :

- Relation = table , Tuple = row
- Attribute = column , Domain = allowed values
- Degree = no. of columns, Cardinality = rows

Integrity constraints :-

- Domain : each value from its domain
- Key : candidate keys stay unique
- Entity : primary key is NOT null
- Referential : FK matches some PK or is null

Concept :-

Entity integrity keeps every row identifiable, and referential integrity keeps links between tables valid. Break these and the data becomes inconsistent.

RELATIONAL ALGEBRA - I

Idea :-

Relational algebra is a procedural query language. Each operator takes relations and returns a relation, so we can chain them to build any query.

Unary operators :

σ (select) rows

a filter (WHERE)

π (project) columns

removes dups

$\sigma_{\text{salary} > 50000}(\text{Emp})$
 $\pi_{\text{name, dept}}(\text{Emp})$

Set operators (need same schema) :

→ Union $\cup$, Intersection, Minus $-$

→ these two relations must be union-compatible

$A \times B$ = cartesian product

every pair of rows

RELATIONAL ALGEBRA - 2

Join :-

A join pairs rows from two relations that satisfy a condition. It is a cartesian product followed by a select, but done together for speed.

→ Theta join : any condition ($<$, $>$, $=$)

→ Equi join : condition uses $=$ only

→ Natural join : equi join on common attrs

```
Emp JOIN[Emp.did = Dept.did] Dept
Emp * Dept           -- natural join
```

Division :-

Division answers for-all queries, like students who took ALL courses. R / S gives values in R matched with every row of S .

ρ (rename) : $\rho(X)(R)$

alias a relation

RELATIONAL CALCULUS

Idea :-

Calculus is a declarative (non-procedural) query language. You state WHAT rows you want, not HOW to get them. It is the theory behind SQL.

Tuple calculus (TRC) :-

Variables range over whole tuples. You write a condition on a tuple variable t and return the tuples that satisfy it.

$$\{ t \mid \text{Emp}(t) \text{ AND } t.\text{sal} > 50000 \}$$

Domain calculus (DRC) :-

Variables range over single attribute values (domains), not whole tuples. Both forms have the same power as safe relational algebra.

$$\{ \langle n, s \rangle \mid \langle n, s \rangle \text{ in Emp AND } s > 50000 \}$$

→ a query must be SAFE (finite answer)

SQL : DDL & DML

DDL (structure) :-

Data Definition Language builds and changes the schema itself, like creating tables and constraints. These commands auto-commit.

```
CREATE TABLE Emp (  
    eid INT PRIMARY KEY,  
    name VARCHAR(30) NOT NULL,  
    did INT REFERENCES Dept(did) );
```

DML (data) :-

Data Manipulation Language works on the rows : insert, update, delete and the all-important select.

```
INSERT INTO Emp VALUES (1, 'Ravi', 10);  
UPDATE Emp SET did=20 WHERE eid=1;  
DELETE FROM Emp WHERE did IS NULL;
```

SQL : JOINS

Concept :-

A join query combines rows of two tables on a matching column. The join type decides which unmatched rows are kept.

```
SELECT e.name, d.dname  
FROM Emp e JOIN Dept d  
ON e.did = d.did;
```

Outer joins keep unmatched rows :

→ LEFT : all left rows, NULLs on right

→ RIGHT : all right rows, NULLs on left

→ FULL : all rows from both sides

Trap :-

An inner join drops rows that have no match. If a department has no employees it vanishes, so use a LEFT join when you must keep every department.

SQL : SUBQUERIES

Nested subquery :-

A subquery in the WHERE clause runs first and feeds its result to the outer query. It runs once and is independent of the outer row.

```
SELECT name FROM Emp
WHERE did IN (SELECT did FROM Dept
              WHERE loc = 'Delhi');
```

Correlated subquery :-

A correlated subquery uses a value from the outer row, so it runs once per outer row. It is slower but very powerful.

```
SELECT name FROM Emp e
WHERE sal > (SELECT AVG(sal) FROM Emp
              WHERE did = e.did);
```

→ look for the outer alias inside : correlated

SQL : GROUP BY & AGG

Aggregate Functions :-

→ COUNT, SUM, AVG, MIN, MAX

Aggregates squeeze many rows into one value. With GROUP BY they give one value per group, like the average salary per department.

```
SELECT did, AVG(sal)
FROM Emp
GROUP BY did
HAVING COUNT(*) > 5;
```

WHERE vs HAVING :-

→ WHERE filters rows BEFORE grouping

→ HAVING filters groups AFTER grouping

You cannot use an aggregate in WHERE. Any column in the SELECT that is not inside an aggregate must appear in the GROUP BY. NULLs are skipped by aggregates, except COUNT(*).

SQL : VIEWS & TRIGGERS

View :-

A view is a virtual table defined by a stored query. It shows fresh data each time and is used for security and to simplify complex joins.

```
CREATE VIEW HighPaid AS  
SELECT name, sal FROM Emp  
WHERE sal > 80000;
```

→ a view stores the query, NOT the rows

Trigger :-

A trigger is code that fires automatically on an insert, update or delete. It is used to enforce rules or keep an audit log.

```
CREATE TRIGGER logpay  
AFTER UPDATE ON Emp  
FOR EACH ROW ...
```

FUNCTIONAL DEPENDENCY

Definition :-

An FD $X \rightarrow Y$ means each value of X fixes exactly one value of Y . It captures a real-world rule and drives all of normalization.

$X \rightarrow Y : X \text{ decides } Y$

$X = \text{determinant}$

Armstrong's axioms :-

→ Reflexivity : $Y \text{ in } X \Rightarrow X \rightarrow Y$

→ Augmentation : $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$

→ Transitivity : $X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$

Derived rules :

→ Union : $X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$

→ Decomposition : $X \rightarrow YZ \Rightarrow X \rightarrow Y, X \rightarrow Z$

Sound & complete :-

Armstrong's axioms are sound, they give only true FDs, and complete, they give every FD that the set implies. That is why closures are reliable.

CLOSURE & CANDIDATE KEY

Attribute closure :-

The closure X^+ is every attribute that X can determine. If X^+ is all attributes, then X is a super key. Solved example :

$R(A, B, C, D, E)$
FDs: $A \rightarrow B, B \rightarrow C, CD \rightarrow E$
 $A^+ = \{A\}$
 $\quad = \{A, B\} \quad (A \rightarrow B)$
 $\quad = \{A, B, C\} \quad (B \rightarrow C)$
 $A^+ = \{A, B, C\} \rightarrow A$ is NOT a key

Candidate key :-

Try $\{A, D\}$. It closes to everything, and neither A nor D alone does, so it is minimal.

$\{A, D\}^+ = \{A, B, C, D, E\} = R$
 $\Rightarrow \{A, D\}$ is a CANDIDATE KEY

$\rightarrow$ attrs on no RHS (like A, D) must be in every key

EQUIVALENCE OF FD SETS

Idea :-

Two FD sets F and G are equivalent if they imply the same FDs, that is, they have the same closure. We check this with attribute closures, not by comparing them directly.

Test $F = G$:

→ F covers G : every FD of G holds under F

→ G covers F : every FD of F holds under G

→ both true $\Rightarrow F$ and G are equivalent

Canonical / minimal cover :-

A minimal cover is a smallest FD set equivalent to F .

Make every right side a single attribute, drop extra attributes from left sides, then remove redundant FDs.

→ single attribute on each RHS

→ no redundant attribute on any LHS

→ no redundant FD in the whole set

NORMALIZATION : 1-2-3 NF

Why :-

Normalization splits tables to remove redundancy and update anomalies. Each higher form is a stricter rule on the FDs.

1NF :-

→ every cell atomic, no repeating groups

2NF :-

→ 1NF + no partial dependency

No non-prime attribute may depend on just PART of a candidate key. Partial dependency only appears with a composite key.

3NF :-

→ 2NF + no transitive dependency

For every FD $X \rightarrow A$, either X is a super key or A is a prime attribute. 3NF is always achievable while preserving dependencies.

BCNF + DECOMPOSITION

BCNF :-

Boyce-Codd normal form is stricter than 3NF. For every FD $X \rightarrow A$ the left side X must be a super key. No prime-attribute exception is allowed.

Solved BCNF check :

$R(A, B, C)$ FDs: $A \rightarrow B, B \rightarrow C$
 $A^+ = ABC \rightarrow A$ is the key
 $B \rightarrow C : B^+ = BC, B$ not superkey
 $\Rightarrow$ VIOLATES BCNF (decompose on B)
 $R_1(B, C) \quad R_2(A, B)$

BCNF vs 3NF :-

BCNF removes all redundancy from FDs but may LOSE dependency preservation. 3NF always keeps dependencies but may keep a little redundancy. That is the classic trade-off.

LOSSLESS & DEP-PRESERVE

Lossless join :-

A decomposition is lossless if joining the parts back gives exactly the original rows, no spurious extra tuples. This is a must-have.

$R(A,B,C)$ split by FD $B \rightarrow C$:

$R_1(B,C)$

share B

$R_2(A,B)$

B is a key of R_1 , so join is LOSSLESS

common attr = key of a part

$\Rightarrow$ lossless

Dependency preserving :-

The decomposition preserves dependencies if every original FD can be checked on a single small table, with no join needed. This keeps constraint checking cheap.

→ lossless : compulsory for correctness

→ dep-preserving : nice, but BCNF may fail it

HIGHER NORMAL FORMS

4NF :-

Fourth normal form removes multi-valued dependencies. An MVD $X \twoheadrightarrow Y$ means Y is a set of values independent of the other attributes. Such sets cause redundant combinations.

4NF : BCNF + no non-trivial MVD

split the MVD

The ladder :-

→ 1NF → 2NF → 3NF → BCNF → 4NF → 5NF

→ each step is stricter than the last

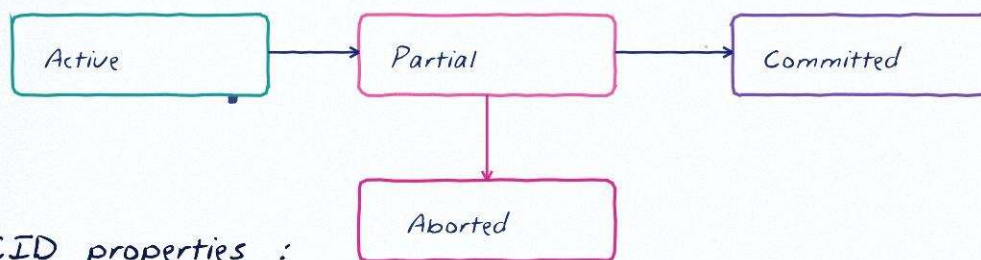
Choosing a form :-

In practice 3NF or BCNF is the target. Go to BCNF for zero FD redundancy; stay at 3NF when you must preserve every dependency. Higher forms matter only when MVDs or join dependencies exist.

TRANSACTIONS : ACID

Definition :-

A transaction is a group of operations treated as one single unit of work. It either fully happens or not at all.



ACID properties :

- Atomicity : all or nothing
- Consistency : rules hold before & after
- Isolation : as if run alone
- Durability : committed data survives crash

Who ensures what :-

Recovery manager gives atomicity and durability, the concurrency manager gives isolation, and the two together keep the database consistent.

SCHEDULES & SERIALIZABILITY

Schedule :-

A schedule is the order in which operations of several transactions run. A serial schedule runs them one after another; a concurrent one interleaves them.

Serializable :-

A concurrent schedule is serializable if its result equals SOME serial order. That is our correctness test for concurrency.

Two kinds :

→ Conflict serializable : swap non-conflicts

→ View serializable : same reads & final writes

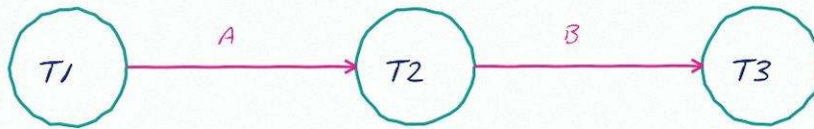
Conflict :-

Two operations conflict if they are from different transactions, use the same data item, and at least one is a write. Conflict serializable implies view serializable, but not the reverse.

PRECEDENCE GRAPH

Method :-

Draw one node per transaction. Add an edge $T_i \rightarrow T_j$ for every conflicting pair where T_i acts first. The schedule is conflict serializable if and only if this graph has NO cycle. ..



Solved example : no cycle : conflict serializable T_1, T_2, T_3

$S: r_1(A) \ w_1(A) \ r_2(A) \ w_2(A) \ r_1(B) \ w_1(B)$
 $r_1(A) - w_2(A): T_1 \rightarrow T_2$
 $w_1(A) - r_2(A): T_1 \rightarrow T_2$
no back edge $\rightarrow$ acyclic

A topological sort of an acyclic graph gives the equivalent serial order, here T_1 then T_2 . A cycle means not serializable.

RECOVERABILITY

Recoverable :-

A schedule is recoverable if a transaction commits only AFTER every transaction whose data it read has committed. Otherwise a commit could depend on a value that later rolls back.

Cascadeless :-

A cascadeless schedule lets a transaction read only committed data. This avoids cascading rollbacks, where aborting one transaction forces many others to abort too.

Nesting (strictest last) :

→ Strict < Cascadeless < Recoverable

→ strict : no read OR write of uncommitted data

Concept :-

Every strict schedule is cascadeless, and every cascadeless schedule is recoverable. Strict schedules make rollback simple, so real systems aim for them.

LOCKING & CONCURRENCY

Two-phase locking (2PL) :-

Each transaction first only acquires locks (growing phase), then only releases them (shrinking phase). 2PL guarantees conflict serializability.

→ Basic 2PL : growing then shrinking

→ Strict 2PL : hold X-locks till commit

→ Rigorous 2PL : hold ALL locks till commit

Deadlock :-

A deadlock is a cycle of transactions each waiting for a lock the next one holds. We prevent it with wait-die or wound-wait, or detect it with a wait-for graph.

Timestamp ordering :-

Each transaction gets a timestamp. Reads and writes must follow that order, using read and write timestamps on each item; a rule violation forces a rollback. It is deadlock-free.

RECOVERY

Log-based recovery :-

The system writes every change to a log first, following the write-ahead rule, so it can redo or undo after a crash. The log is the single source of truth for recovery.

$\langle T, X, \text{old}, \text{new} \rangle$

a log record

Deferred vs immediate :-

→ Deferred : write to DB only after commit (REDO)

→ Immediate : write early, so UNDO + REDO

Checkpoint :-

A checkpoint flushes the log and buffers to disk and marks a safe point. On restart we only scan the log after the last checkpoint, so recovery is fast.

→ before checkpoint & committed : ignore

→ committed after checkpoint : REDO

→ not committed : UNDO

INDEXING & FILE ORG

Index :-

An index is a small sorted structure of (key, pointer) that speeds up search. It works like a book index, so we avoid scanning every row.

By key type :

- Primary : on ordered PK, sparse allowed
- Clustering : on a non-key ordering field
- Secondary : on any other field, must be dense

Dense vs sparse :-

- Dense : one entry per record
- Sparse : one entry per block

A file can be ordered on only ONE field, so only that field gets a sparse or clustering index. Every other index must be dense and secondary.

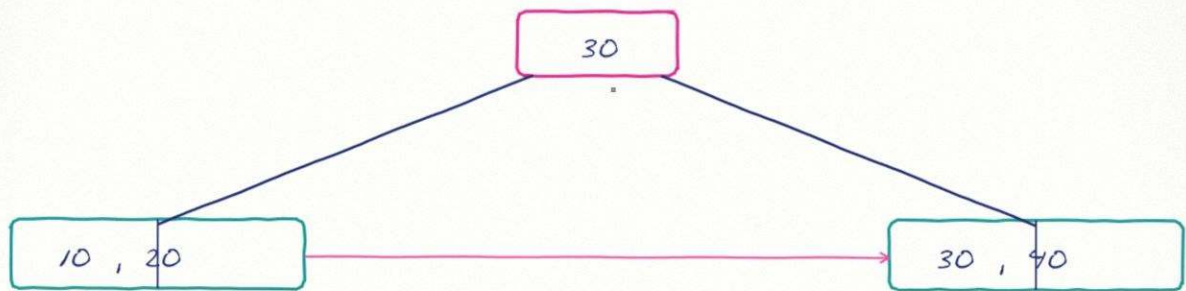
Multi-level :-

A large index is itself indexed, giving levels that shrink search to a few block reads. This idea leads straight to the B+ tree.

B-TREE & B+ TREE

Idea :-

A B+ tree is a balanced multi-level index. All leaves sit at the same depth, so every search costs the same few disk reads.



leaves hold ALL keys + are chained (range scan)

B-tree vs B+ tree :

→ B-tree : keys + data in ALL nodes

→ B+ tree : data only in leaves

→ B+ leaves are linked : fast range scan

Search / insert :-

Search walks root to leaf comparing keys, in log time. Insert puts the key in a leaf; if the node overflows it splits and pushes a separator key up, which may split the parent.

B+ TREE ORDER : SOLVED

Setup :-

Find the order of an internal node and the leaf capacity.
An internal node of order n holds n block-pointers and $n-1$ keys.

$$\begin{aligned} \text{block} &= 512 \text{ B, key } K = 10 \text{ B} \\ \text{block-ptr } P &= 8, \text{ record-ptr } Pr = 8 \\ \text{Internal: } n * P + (n-1) * K &\leq 512 \\ 8n + 10(n-1) &\leq 512 \\ 18n &\leq 522 \rightarrow n = 29 \\ \text{Leaf: } n_l * (K + Pr) + P &\leq 512 \\ 18 * n_l + 8 &\leq 512 \rightarrow n_l = 28 \end{aligned}$$

Read it as :-

So each internal node fans out to 29 children and each leaf stores 28 records. High fan-out keeps the tree short, which is exactly why databases use B+ trees on disk.

HASHING

Idea :-

Hashing maps a key straight to a bucket with a hash function, giving near constant-time lookup. It is ideal for equality search but useless for range queries.

Static hashing :-

The number of buckets is fixed. If data grows, buckets overflow and we chain overflow pages, which slowly hurts performance.

Dynamic hashing :-

→ Extendible : a directory doubles on need

→ Linear : buckets split one at a time

Dynamic hashing grows and shrinks the bucket count as data changes, so it avoids long overflow chains. Extendible hashing uses the first few bits of the hash to index a directory.

→ hashing : great for $=$, bad for $<$ and $>$

QUICK REFERENCE SHEET

Normal forms :-

- 1NF : atomic values, no repeating groups
- 2NF : 1NF + no partial dependency
- 3NF : 2NF + no transitive dependency
- $(X \rightarrow A : X \text{ superkey OR } A \text{ prime})$
- BCNF : every $X \rightarrow A$ has $X = \text{superkey}$
- 4NF : BCNF + no non-trivial MVD

Serializability :-

- conflict : same item, diff txn, one write
- precedence graph acyclic $\Leftrightarrow$ conflict serial
- conflict SR $\Rightarrow$ view SR (not reverse)
- strict $<$ cascadeless $<$ recoverable
- 2PL $\Rightarrow$ conflict serializable schedule

COMMON MISTAKES

Watch out :-

- candidate key = MINIMAL super key; not any
- attrs on no RHS must be in every key
- partial dependency needs a composite key
- BCNF can lose dependency preservation
- HAVING for groups, WHERE for rows
- COUNT(*) counts NULLs, other aggs skip them
- conflict serial $\Rightarrow$ view, NOT the other way
- file ordered on ONE field only (one sparse)
- secondary index must be DENSE
- B+ tree stores data in leaves only
- natural join drops unmatched rows

GATE TIPS & RECALL

High-weightage topics :-

- FDs, closure, candidate keys
- normalization : 3NF & BCNF
- conflict serializability, 2PL
- SQL joins, subqueries, aggregates
- B+ tree order & node counting

Quick checks :-

- $X^+ = \text{all attrs} \Rightarrow X \text{ is a super key}$
- lossless : common attr is a key of a part
- timestamp ordering is deadlock-free
- higher fan-out $\Rightarrow$ shorter B+ tree

All the best !